

ARCHITECTURE

LLMProvider Module - Architecture

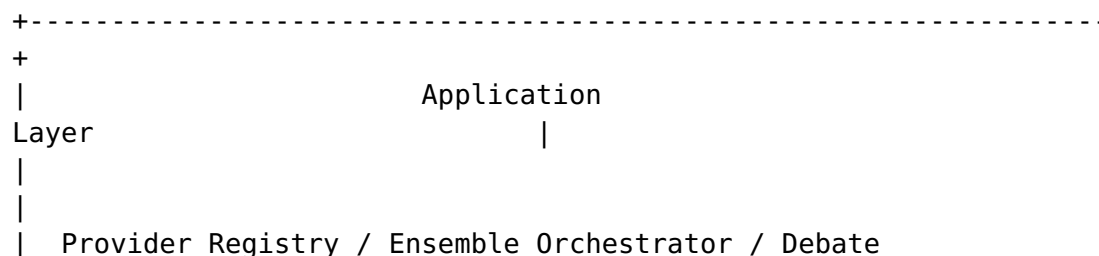
Module: digital.vasic.llmprovider **Version:** 1.0.0 **Last Updated:** March 2026

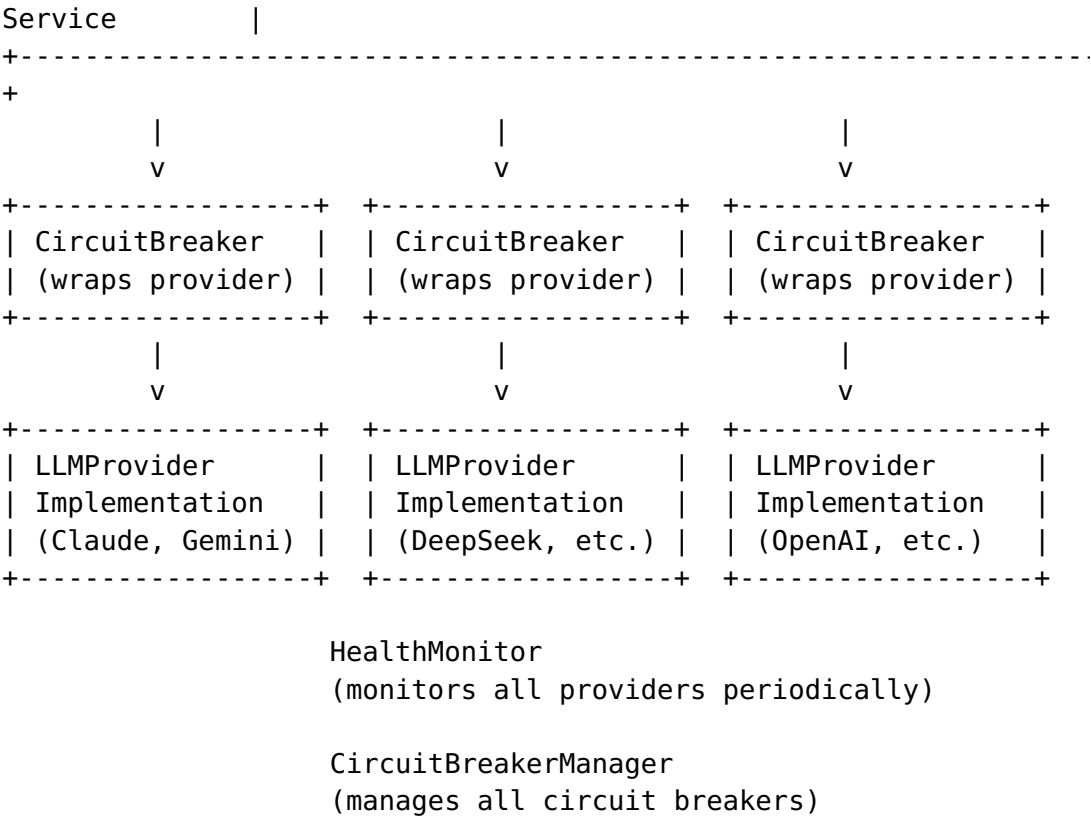
Design Philosophy

The LLMProvider module provides **generic, reusable abstractions** for building fault-tolerant LLM provider integrations. It is designed to:

1. **Define a stable interface** -- the LLMProvider interface is the contract all providers implement.
 2. **Provide fault tolerance** -- circuit breakers prevent cascading failures across providers.
 3. **Enable observability** -- health monitors track provider availability in real time.
 4. **Handle transient errors** -- retry logic with exponential backoff and jitter.
 5. **Support lazy initialization** -- expensive provider setup is deferred until first use.
 6. **Be thread-safe** -- all components are designed for concurrent use.
-

High-Level Architecture



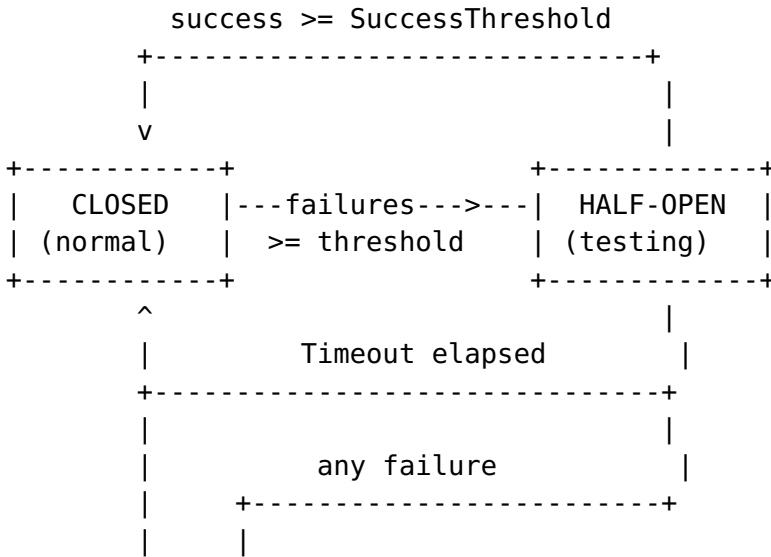


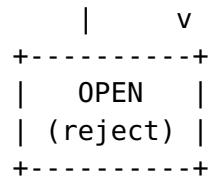
Component Details

Circuit Breaker Pattern

The circuit breaker prevents cascading failures when an LLM provider becomes unhealthy. It wraps the LLMProvider interface transparently, so callers interact with it as if it were a regular provider.

State Machine:





Closed State: All requests pass through to the underlying provider. Consecutive failures are tracked. When consecutive failures reach `FailureThreshold`, the circuit transitions to Open.

Open State: All requests are immediately rejected with `ErrCircuitOpen`. After `Timeout` elapses, the circuit transitions to Half-Open.

Half-Open State: A limited number of requests (`HalfOpenMaxRequests`) are allowed through. If `SuccessThreshold` consecutive successes occur, the circuit transitions back to Closed. Any single failure returns it to Open.

Streaming Support: The `CompleteStream` method wraps the response channel to track success or failure. An empty stream (no responses received) is treated as a failure.

Listener Notifications: State change listeners are notified in separate goroutines with a 5-second timeout to prevent slow listeners from blocking state transitions. Listeners are copied from the map before notification to avoid holding the lock during callbacks. A maximum of 100 listeners is enforced per circuit breaker to prevent memory leaks.

Health Monitor

The health monitor periodically checks the health of registered providers and tracks their status over time.

Status Transitions:

```

UNKNOWN --- (success) ---> HEALTHY
UNKNOWN --- (failure) ---> UNKNOWN (stays until threshold)
HEALTHY --- (failure) ---> DEGRADED
DEGRADED -- (more failures >= threshold) ---> UNHEALTHY
UNHEALTHY - (success count >= threshold) ---> HEALTHY
DEGRADED -- (success count >= threshold) ---> HEALTHY

```

Concurrent Health Checks: All registered providers are checked concurrently using goroutines with a `sync.WaitGroup`. Each check has an individual timeout (configurable, default 10s).

Manual Recording: In addition to periodic checks, the `RecordSuccess` and `RecordFailure` methods allow external systems (such as the circuit breaker) to contribute to health status without waiting for the next check cycle.

Aggregate Health: The `GetAggregateHealth` method provides a system-wide health summary with overall status determined by the proportion of healthy, degraded, and unhealthy providers.

Retry Logic

The retry module provides exponential backoff with jitter for handling transient failures in LLM API calls.

Backoff Formula:

```
delay(attempt) = min(InitialDelay * Multiplier^(attempt-1),
MaxDelay)
jittered_delay = delay +/- (delay * JitterFactor * random)
```

Retryable Conditions:

- HTTP status codes: 429 (Too Many Requests), 500, 502, 503, 504
- Network errors (connection refused, timeout, DNS errors)

Non-Retryable Conditions:

- `context.Canceled` -- caller explicitly cancelled
- `context.DeadlineExceeded` -- overall timeout reached
- HTTP 4xx errors (except 429) -- client errors are not transient

Context Integration: The retry loop checks `ctx.Done()` before each attempt and during the backoff wait, ensuring prompt cancellation when the caller's context expires.

RetryableHTTPClient: A convenience wrapper around `http.Client` that automatically applies retry logic. Requests are cloned for each attempt to handle body consumption.

Lazy Provider (Conceptual)

The lazy provider pattern defers expensive initialization (API key validation, model listing, connection setup) until the first actual request. This is critical for `HelixAgent`'s startup, where 43 providers are registered but only a subset may be used.

Key Characteristics:

- Uses `sync.Once` for thread-safe initialization
- Configurable initialization timeout and retry attempts

- Optional event bus integration for publishing provider lifecycle events (initialized, failed)
 - Falls back gracefully if initialization fails
-

Thread Safety Model

Component	Synchronization	Notes
CircuitBreaker	sync.RWMutex	Lock held during state transitions; listeners notified outside lock
CircuitBreakerManager	sync.RWMutex	Protects the breakers map; individual breakers have their own locks
HealthMonitor	sync.RWMutex	Provider map and health map protected; checks run concurrently
RetryConfig	Immutable	No synchronization needed after construction
RetryableHTTPClient	Immutable config	Each request cloned per attempt; no shared mutable state

Lock Ordering: The circuit breaker copies the listener list before notification to avoid holding the lock during potentially slow callbacks. The Reset method explicitly unlocks before notifying listeners to prevent deadlocks.

Integration with HelixAgent

In HelixAgent, the LLMProvider module is used as follows:

1. **Provider Registry** (internal/services/provider_registry.go): Registers 43 LLM providers, each wrapped with a circuit breaker from the CircuitBreakerManager.
2. **Startup Verification** (internal/verifier/): Uses HealthMonitor to verify provider availability during startup, scoring and ranking providers based on verification results.
3. **Ensemble Orchestration** (internal/llm/ensemble.go): Routes requests to available providers, falling back through the chain when circuit breakers trip.

4. **Debate Service** (internal/services/debate_service.go): Uses circuit breaker status to select healthy providers for multi-LLM debate rounds.
-

File Structure

```
LLMProvider/
  provider.go          -- LLMProvider interface definition
  circuit_breaker.go   -- CircuitBreaker,
CircuitBreakerManager, listeners
  health_monitor.go    -- HealthMonitor, ProviderHealth,
AggregateHealth
  retry.go             -- RetryConfig, ExecuteWithRetry,
RetryableHTTPClient
  types.go             -- (Types moved to models package)
  doc.go               -- Package documentation
  *_test.go            -- Comprehensive test coverage
```